

Lab 2: Long-Range Memory

Copy the first bit of a long sequence: vanilla RNN versus LSTM.

Work in a **Jupyter** notebook with **PyTorch**. Do notes **2.1–2.3** first. CPU is enough.

Upload HTML to Canvas when done.

The task is **copy the first bit at the end of a long sequence**. A vanilla RNN should fail as T grows; an LSTM should keep the bit.

```
import torch
from torch import nn
from torch.nn.utils import clip_grad_norm_

def batch(T, n=64):
    x = torch.randint(0, 2, (n, T, 1)).float()
    y = x[:, 0, :]          # label = first bit
    return x, y
```

1. A linear baseline (no memory)

1. Build `nn.Linear(1, 1)` and train it on `x[:, -1, :]` (the **last** bit) to predict `y`. Accuracy should be ≈ 0.5 . Why?
2. In one sentence: this model has no path from position 0 to the output except through whatever you fed it.

2. Vanilla RNN versus LSTM

Use `T = 25` first.

```
class SeqModel(nn.Module):
    def __init__(self, kind="rnn"):
        super().__init__()
        if kind == "rnn":
            self.rnn = nn.RNN(1, 16, batch_first=True)
        else:
            self.rnn = nn.LSTM(1, 16, batch_first=True)
        self.out = nn.Linear(16, 1)

    def forward(self, x):
        h, _ = self.rnn(x)
        return self.out(h[:, -1, :])
```

1. Train each model 400 steps, Adam `lr=1e-2`, `BCEWithLogitsLoss`. Print accuracy on a fresh batch.
 2. Repeat for `T = 5, 15, 40`. Make a table: T versus RNN accuracy versus LSTM accuracy.
 3. For the vanilla RNN at `T=40`, print `clip_grad_norm_(model.parameters(), 1e9)` once after a backward pass (no clip). Is the norm tiny or huge?
-

3. What you should see

Short T : both work. Long T : RNN near chance, LSTM above chance. If both fail, train longer or widen the hidden size to 32. If the LSTM is also chance at $T = 40$, you likely used the last input as the label by accident.

Write four sentences: what the hidden state had to store, why tanh along the chain hurts, what the forget gate is for, and why Week 3 will stop using this loop.